

Array Practice — Set 2 (Two-Pointer & HashMap)

Personal Notes — Solved Practice Set

This is the second practice set on arrays — moving past basic traversal into more powerful techniques. The problems here introduce two patterns that show up everywhere in DSA: the TWO-POINTER technique and the FREQUENCY MAP (HashMap) pattern.

Master these two patterns and a huge category of array and string problems becomes approachable: reversing, palindromes, sliding window, duplicates, anagrams, most-frequent, missing-number — they all build on these ideas.

Two new patterns introduced: (1) Two-Pointer — use two index variables (start and end) moving toward each other or in the same direction. Cuts many $O(n^2)$ problems down to $O(n)$. (2) Frequency Map — use a `HashMap<Element, Count>` to record how many times each value appears in one pass.

Problem 1: Reverse an Array (Two-Pointer)

Reverse the elements of an array IN PLACE — without creating a second array.

Solution

```
class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};

        int start = 0;
        int end = arr.length - 1;

        while (start < end) {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }

        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

Explanation

Place two pointers — one at the start, one at the end. Swap their values, then move both inward by one. Stop when they meet in the middle.

Dry Run for {1, 2, 3, 4, 5}

```
Start:  [1, 2, 3, 4, 5]   start=0  end=4
         ^               ^

Step 1: swap arr[0] and arr[4]
        [5, 2, 3, 4, 1]   start=1  end=3
           ^             ^

Step 2: swap arr[1] and arr[3]
        [5, 4, 3, 2, 1]   start=2  end=2
           ^
        start == end → stop

Result: [5, 4, 3, 2, 1]
```

Why two-pointer beats a naive solution: A naive reverse uses a second array (extra $O(n)$ space). Two-pointer reverses in $O(n)$ time with $O(1)$ extra space — way more efficient for large arrays.

Edge cases to think about

- Empty array — while loop never runs, nothing happens. ✓ safe
- Single element — start = 0, end = 0, condition fails immediately. ✓ safe
- Even length — pointers cross over without meeting (start > end). Loop ends correctly.
- Odd length — pointers meet at the middle element. That element doesn't swap with itself.

Problem 2: Shift Array Right by 1 Position

Shift every element one position to the right. The LAST element wraps around to position 0.

Solution

```
class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        int temp = arr[arr.length - 1];

        for (int i = arr.length - 1; i > 0; i--) {
            arr[i] = arr[i - 1];
        }
        arr[0] = temp;

        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

```

    }
}
}

```

Explanation

This is a RIGHT ROTATION by 1. Three steps:

- Save the last element (it would otherwise be lost when overwritten)
- Shift everything one position right — starting from the end, copy `arr[i-1]` into `arr[i]`
- Put the saved value at index 0

Dry Run for {1, 2, 3, 4, 5}

```

Start:   [1, 2, 3, 4, 5]   temp = 5   (saved)

i = 4:   arr[4] = arr[3] = 4   → [1, 2, 3, 4, 4]
i = 3:   arr[3] = arr[2] = 3   → [1, 2, 3, 3, 4]
i = 2:   arr[2] = arr[1] = 2   → [1, 2, 2, 3, 4]
i = 1:   arr[1] = arr[0] = 1   → [1, 1, 2, 3, 4]

arr[0] = temp = 5               → [5, 1, 2, 3, 4]

Result: [5, 1, 2, 3, 4]

```

Why we loop BACKWARDS: If you looped forward ($i = 0, 1, 2, \dots$), each step would OVERWRITE `arr[i+1]` before you could shift it. By going backwards (from end to start), you always copy from a position that hasn't been touched yet. Direction matters for in-place shifts.

Bonus: Shift LEFT by 1

```

int temp = arr[0];
for (int i = 0; i < arr.length - 1; i++) {
    arr[i] = arr[i + 1];
}
arr[arr.length - 1] = temp;

```

Same idea, but loop forward and save the first element instead.

Problem 3: Print Extremes in Alternate Order

Print elements like a zig-zag from outside in: first → last → second → second-last → ... until they meet.

Solution

```

class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5, 6};
        int start = 0;
        int end = arr.length - 1;
    }
}

```

```

        while (start <= end) {
            if (start == end) {
                System.out.print(arr[start] + " ");
                break;
            } else {
                System.out.print(arr[start] + " ");
                start++;
                System.out.print(arr[end] + " ");
                end--;
            }
        }
    }
}

```

Explanation

Another two-pointer problem. Print start, then end, then move both pointers inward. The special case is when `start == end` (odd-length array) — print that one element once and stop.

Dry Run for {1, 2, 3, 4, 5, 6}

```

Start:  [1, 2, 3, 4, 5, 6]   start=0  end=5

Print arr[0]=1, arr[5]=6 → output so far: 1 6
                        start=1  end=4

Print arr[1]=2, arr[4]=5 → output so far: 1 6 2 5
                        start=2  end=3

Print arr[2]=3, arr[3]=4 → output so far: 1 6 2 5 3 4
                        start=3  end=2 → loop ends

Final output:  1 6 2 5 3 4

```

Dry Run for odd length {1, 2, 3, 4, 5}

```

start=0 end=4:  print 1, 5 → "1 5"   start=1 end=3
start=1 end=3:  print 2, 4 → "1 5 2 4" start=2 end=2
start=2 end=2:  start == end → print 3, break

Final output:  1 5 2 4 3

```

Pattern recognition: Any problem that says "alternate from extremes" or "outside-in zig-zag" is likely a two-pointer problem. Use start and end, decide what to do on each step, and remember to handle the meeting point (odd-length case).

Problem 4: Build a Frequency Map

Count how many times each value appears in the array, using a HashMap.

Solution

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        int[] arr = {5, 2, 3, 2, 3, 1, 2, 9};

        HashMap<Integer, Integer> freq = new HashMap<>();
        for (int num : arr) {
            freq.put(num, freq.getOrDefault(num, 0) + 1);
        }

        for (int i : freq.keySet()) {
            System.out.println(i + " -> " + freq.get(i));
        }
    }
}
```

Explanation

This is the FREQUENCY MAP pattern — one of the most useful patterns in all of DSA. Here's how it works:

- Walk through the array once
- For each number, look up its current count (or 0 if not seen yet)
- Add 1 and store it back

`getOrDefault(num, 0)` is the trick that makes this clean — it returns the existing count, or 0 if the key isn't there yet.

Dry Run for {5, 2, 3, 2, 3, 1, 2, 9}

```
num=5 → freq = {5: 1}
num=2 → freq = {5: 1, 2: 1}
num=3 → freq = {5: 1, 2: 1, 3: 1}
num=2 → freq = {5: 1, 2: 2, 3: 1}
num=3 → freq = {5: 1, 2: 2, 3: 2}
num=1 → freq = {5: 1, 2: 2, 3: 2, 1: 1}
num=2 → freq = {5: 1, 2: 3, 3: 2, 1: 1}
num=9 → freq = {5: 1, 2: 3, 3: 2, 1: 1, 9: 1}
```

Output (order may vary — HashMap is unordered):

```
5 -> 1
2 -> 3
3 -> 2
```

```
1 -> 1
9 -> 1
```

Want ordered output? HashMap doesn't preserve order. Use LinkedHashMap to preserve INSERTION order, or TreeMap to keep keys in SORTED order. Just swap `new HashMap<>()` with `new LinkedHashMap<>()` or `new TreeMap<>()` — same code otherwise.

Bonus: Using `entrySet()` for cleaner iteration

```
for (Map.Entry<Integer, Integer> entry : freq.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
```

`entrySet()` gives you both key and value in one shot. Marginally faster than calling `freq.get(i)` inside a `keySet()` loop, because it avoids the extra lookup.

Problem 5: Element with Max & Min Frequency

Find which element appears the MOST times and which appears the FEWEST times.

Solution

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        int[] arr = {5, 2, 3, 2, 3, 1, 2, 9};

        HashMap<Integer, Integer> freq = new HashMap<>();

        int maxFreq = -1;
        int minFreq = Integer.MAX_VALUE;
        int maxFreqKey = -1;
        int minFreqKey = -1;

        // Step 1: build frequency map
        for (int num : arr) {
            freq.put(num, freq.getOrDefault(num, 0) + 1);
        }

        // Step 2: scan the map to find extremes
        for (int i : freq.keySet()) {
            int count = freq.get(i);
            if (count > maxFreq) {
                maxFreq = count;
                maxFreqKey = i;
            }
            if (count < minFreq) {
                minFreq = count;
            }
        }
    }
}
```

```

        minFreqKey = i;
    }
}

for (int i : freq.keySet()) {
    System.out.println(i + " -> " + freq.get(i));
}
System.out.println("Maximum frequency element: " + maxFreqKey + "
-> " + maxFreq);
System.out.println("Minimum frequency element: " + minFreqKey + "
-> " + minFreq);
}
}

```

Explanation

This is a TWO-PASS solution that combines the frequency map (Problem 4) with the max/min tracking pattern (from Set 1, Problem 4).

- Pass 1 — Build the frequency map (same as Problem 4)
- Pass 2 — Walk the map and track BOTH the max and the min count, along with the keys that produced them

Why these initial values?

```

int maxFreq = -1;           // any real count will beat -1
int minFreq = Integer.MAX_VALUE; // any real count will be smaller

```

Initializing maxFreq to -1 and minFreq to Integer.MAX_VALUE GUARANTEES that the first real count overwrites them. This is the standard "sentinel value" trick.

Dry Run for {5, 2, 3, 2, 3, 1, 2, 9}

Frequency map: {5: 1, 2: 3, 3: 2, 1: 1, 9: 1}

Pass 2 — walking the map (order may vary):

key=5	count=1	1 > -1? YES → maxFreq=1, maxKey=5
		1 < MAX_VALUE? YES → minFreq=1, minKey=5
key=2	count=3	3 > 1? YES → maxFreq=3, maxKey=2
		3 < 1? no
key=3	count=2	2 > 3? no
		2 < 1? no
key=1	count=1	1 > 3? no
		1 < 1? no
key=9	count=1	1 > 3? no
		1 < 1? no

Result:

Max frequency: 2 -> 3

Min frequency: 5 -> 1

Edge case: ties in min frequency In our example, 5, 1, and 9 all appear once. The code picks whichever key it visits FIRST (HashMap order is not guaranteed). If you specifically want "the smallest key with min frequency", add another condition or use TreeMap to make the iteration deterministic.

Patterns You're Now Using

These 5 problems introduce two reusable patterns that show up everywhere:

Pattern 1: Two-Pointer

```
int start = 0;
int end = arr.length - 1;

while (start < end) {          // or start <= end
    // process arr[start] and arr[end]
    // ... swap, print, compare, etc.
    start++;
    end--;
}
```

Used in: reverse, palindrome check, extreme zig-zag, two-sum on sorted arrays, finding pairs that sum to a target.

Pattern 2: Frequency Map

```
HashMap<Integer, Integer> freq = new HashMap<>();
for (int num : arr) {
    freq.put(num, freq.getOrDefault(num, 0) + 1);
}
// freq now holds count of every element
```

Used in: counting occurrences, finding duplicates, anagram check, first non-repeating element, top-K frequent elements, mode of an array.

More Problems — Same Patterns

Two-Pointer Practice

- Check if a string is a palindrome (compare from both ends inward)
- Check if an array is a palindrome (same idea)
- Shift the array LEFT by 1 position
- Rotate the array by K positions (use the reverse trick: reverse all, reverse first K, reverse rest)
- Two-Sum on a SORTED array — find a pair that sums to target in O(n)
- Move all zeros to the end (start pointer for non-zeros, end pointer for zeros)
- Reverse only vowels in a string

- Container With Most Water (LeetCode 11)

HashMap Practice

- Find the first non-repeating character in a string
- Check if two arrays are anagrams (same frequency map)
- Find the element that appears more than $N/2$ times (majority element)
- Two-Sum on an UNSORTED array — store seen values in a HashMap as you go
- Find all duplicates in an array
- Group anagrams: cluster strings that are anagrams of each other
- Count the number of distinct elements in a window of size K (sliding window + map)
- Subarray sum equals K

Combined (use BOTH patterns)

- Reverse a string and check if it equals the original (palindrome check)
- Find the longest substring without repeating characters (sliding window + map)
- Given an array, partition it so all elements with frequency > 1 come first

Small Improvements to Your Code

Your solutions all work correctly. Here are a few small refinements professional Java developers tend to make.

1. Prefer enhanced for-loop for simple traversal

```
// Your style – index-based
for (int i = 0; i < arr.length; i++) {
    System.out.print(arr[i] + " ");
}

// Cleaner – for-each (when you don't need the index)
for (int num : arr) {
    System.out.print(num + " ");
}
```

2. Use entrySet() for map iteration

```
// Two lookups per iteration (key, then value)
for (int i : freq.keySet()) {
    System.out.println(i + " -> " + freq.get(i));
}

// One pass – gets key and value together
for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue());
}
```

```
}
```

3. Use Math.min / Math.max for cleaner comparison

Instead of writing out the if-else for tracking max, you can use the Math methods — though it's a minor style preference.

4. Extract reusable logic into methods

When the same pattern shows up multiple times (like printing the array, or building the frequency map), put it in a method:

```
static void printArray(int[] arr) {
    for (int num : arr) System.out.print(num + " ");
    System.out.println();
}

static HashMap<Integer, Integer> buildFreq(int[] arr) {
    HashMap<Integer, Integer> freq = new HashMap<>();
    for (int num : arr) freq.put(num, freq.getOrDefault(num, 0) + 1);
    return freq;
}
```

Key Takeaways

- Two-Pointer is your tool for any "both ends moving inward" problem
- HashMap with getOrDefault is the standard frequency-counting pattern
- When shifting in-place, the LOOP DIRECTION matters — choose carefully
- Save values BEFORE overwriting them (the temp variable trick)
- Initialize maxFreq to -1 and minFreq to Integer.MAX_VALUE for the comparison-tracking pattern
- HashMap doesn't preserve order — use LinkedHashMap or TreeMap when order matters
- Always test edge cases: empty array, single element, all duplicates, length 1 vs 2

Final encouragement: You've moved from "basic array traversal" to "two patterns that solve dozens of real DSA problems." The next big step is the SLIDING WINDOW pattern — which combines what you just learned and is the key to many string and subarray problems. Keep going.